

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Омский государственный технический университет»

Факультет
(институт)

Нефтехимический

Кафедра

«Цифровое управление процессами и системами
нефтегазового комплекса»

ОТЧЕТ
по производственной практике
на тему «Трекер задач»

Студент:

Сероухов Егор Сергеевич

Курс 1. Группа ИСТм-251.

Руководитель:

Гуглина Алла Владимировна

«___»_____2026 г.

Омск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Архитектура системы и стек технологий	4
1.1 Обоснование выбора технологий.....	4
1.2 Контейнеризация и инфраструктура (Docker, Mailpit, pgAdmin) ..	5
2 Программная реализация бэкенд-сервиса	8
2.1 Проектирование базы данных и миграции (Liquibase)	8
2.2 Система безопасности и авторизация (Spring Security + JWT)	9
2.3 Реализация бизнес-логики управления задачами (Records, Switch expressions).....	10
3 Дополнительные сервисы и интеграции.....	11
3.1 Интеграция почтовой подсистемы на базе Mailpit.....	11
3.2 Автоматизация фоновых процессов и планировщик (@Scheduled)	12
3.3 Оркестрация сервисов через Docker Compose	12
4 Клиентское приложение (Frontend).....	14
4.1 Архитектура и стек технологий фронтенда	14
4.2 Механизм аутентификации и работа с JWT.....	14
4.3 Функциональные возможности интерфейса	15
5 Тестирование и развертывание.....	17
5.1 Модульное и интеграционное тестирование (JUnit 5, Mockito) ..	17
5.2 Тестирование	17
5.2.1 Unit-тестирование бизнес-логики (TaskServiceTest, AuthServiceTest).....	17
5.2.2 Интеграционное тестирование API (AuthControllerTest).....	18
5.3 Процесс сборки и контейнеризация (Docker)	19
ЗАКЛЮЧЕНИЕ	20

ВВЕДЕНИЕ

Целью данной работы является разработка распределенного веб-приложения «Трекер задач». Система позволяет пользователям регистрироваться, проходить аутентификацию и управлять персональным списком задач, включая назначение исполнителей и смену статусов. В процессе разработки использовались современные стандарты платформы Java, такие как Java 21 (LTS) и Spring Boot 4.0.6, что позволило применить новые языковые конструкции и улучшить производительность сервиса.

1 Архитектура системы и стек технологий

Система построена по принципу клиент-серверной архитектуры.

Database. Реляционная база данных PostgreSQL 16. Управление схемой данных осуществляется через Liquibase, что гарантирует воспроизводимость структуры на любых устройствах.

Backend. Использует Spring Boot для создания RESTful API. Для обеспечения безопасности применен стандарт JWT (JSON Web Token), что делает систему масштабируемой (stateless).

Frontend. Легковесное Single Page Application (SPA), использующее нативный JavaScript и Bootstrap для оформления.

Инфраструктура. Весь стек развернут в Docker-контейнерах. Для отладки почтовых уведомлений интегрирован сервер Mailpit.

1.1 Обоснование выбора технологий

Для реализации проекта был выбран современный стек технологий, обеспечивающий высокую скорость разработки и надежность.

Spring Boot 4.0.6. Актуальная версия фреймворка, минимизирующая конфигурацию и предоставляющая мощные инструменты для создания REST-сервисов.

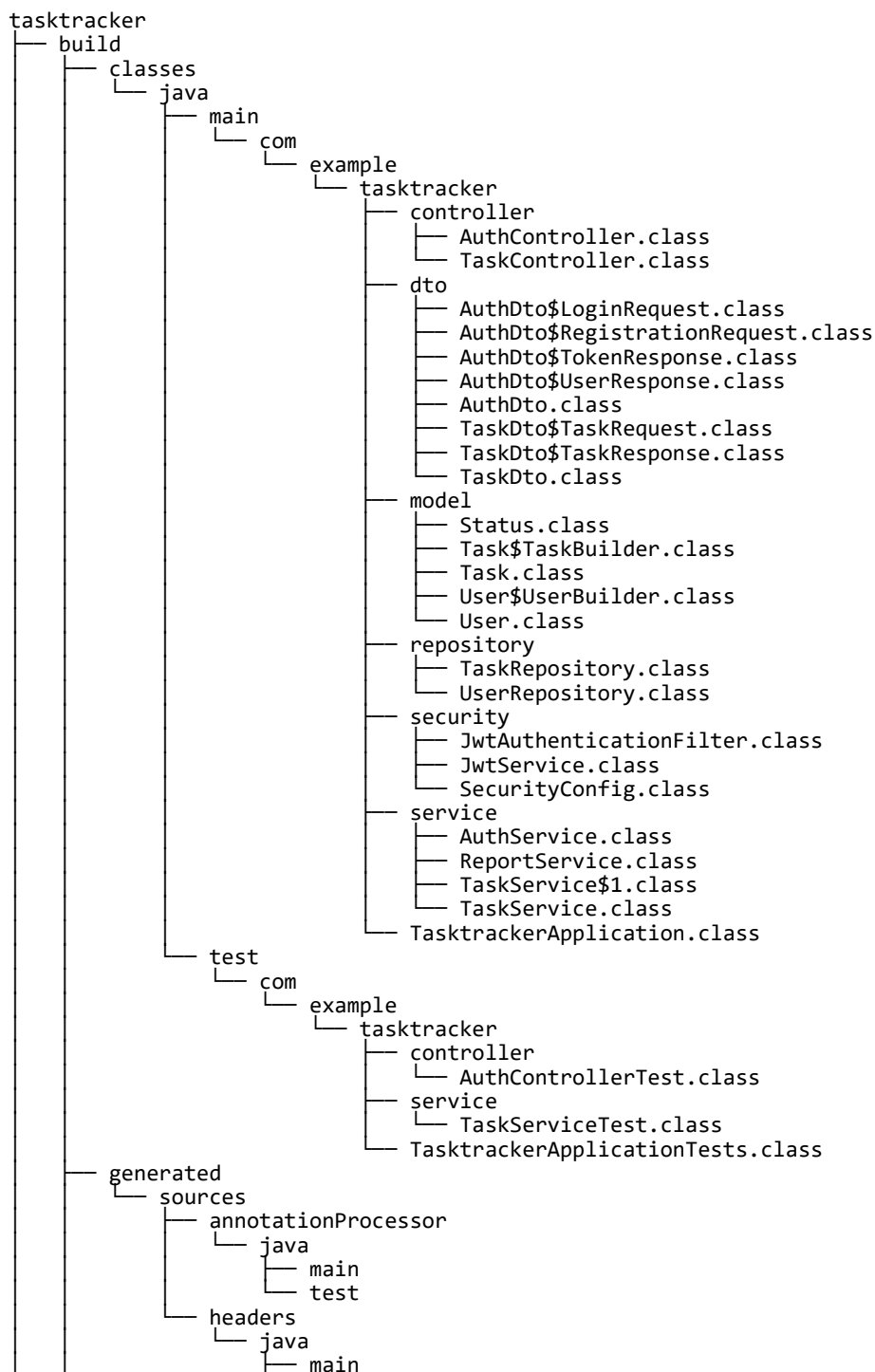
Java 21. Использование стабильной версии с долгосрочной поддержкой (LTS) позволило задействовать современные возможности языка (Records, Switch Expressions).

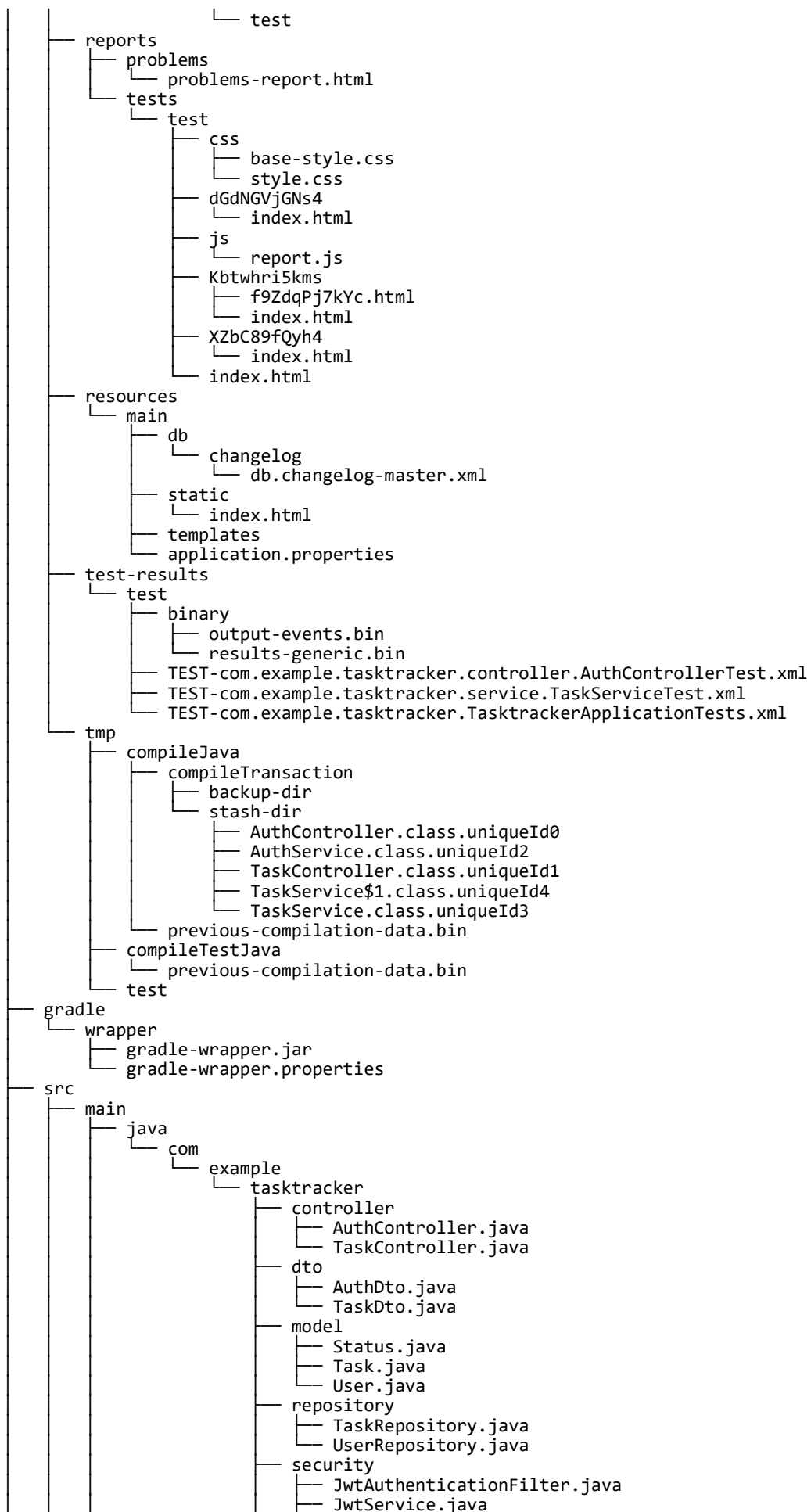
PostgreSQL. Реляционная СУБД, выбранная за надежность и поддержку сложных типов данных.

Bootstrap 5. CSS-фреймворк для быстрого создания адаптивного интерфейса без необходимости глубокой верстки.

1.2 Контейнеризация и инфраструктура (Docker, Mailpit, pgAdmin)

Проект спроектирован с учетом принципов переносимости. Использование Docker Compose позволяет развернуть всю необходимую инфраструктуру (базу данных, почтовый сервер и клиент для управления БД) одной командой. Это исключает проблему развертки системы. Ниже представлено дерево проекта со всеми пакетами (controller, model, service и т.д.).





```

├── SecurityConfig.java
├── service
│   ├── AuthService.java
│   ├── ReportService.java
│   ├── TaskService.java
│   └── TasktrackerApplication.java
├── resources
│   ├── db
│   │   ├── changelog
│   │   └── db.changelog-master.xml
│   ├── static
│   │   └── index.html
│   ├── templates
│   └── application.properties
├── test
│   ├── java
│   │   ├── com
│   │   │   └── example
│   │   │       └── tasktracker
│   │   │           ├── controller
│   │   │           │   └── AuthControllerTest.java
│   │   │           ├── service
│   │   │           │   └── TaskServiceTest.java
│   │   │           └── TasktrackerApplicationTests.java
│   └── test.http
├── build.gradle
├── docker-compose.yml
├── gradlew
├── gradlew.bat
├── HELP.md
├── README.md
└── settings.gradle

```

Листинг кода 1 – Дерево проекта с пакетами

2 Программная реализация бэкенд-сервиса

Бэкенд-часть приложения является ключевым звеном системы, отвечающим за бизнес-логику, обработку запросов и обеспечение целостности данных. Реализация серверной части выполнена на фреймворке Spring Boot с использованием слоистой архитектуры (Controller -> Service -> Repository). Такой подход позволяет изолировать логику обработки HTTP-запросов от манипуляций с базой данных и упрощает тестирование и поддержку кода.

2.1 Проектирование базы данных и миграции (Liquibase)

Для взаимодействия с СУБД PostgreSQL использован стек Spring Data JPA. Описание таблиц реализовано через сущности (Entities) с использованием аннотаций Jakarta Persistence.

UserRepository обеспечивает поиск пользователей по Email для аутентификации.

TaskRepository реализует выборку задач, принадлежащих конкретному владельцу.

Вместо небезопасного механизма автоматического обновления таблиц Hibernate (ddl-auto: update), в проекте применен Liquibase. Все изменения структуры базы данных описаны в XML-файлах миграций (changelogs), что позволяет отслеживать историю изменений БД и гарантирует идентичность схем данных на сервере разработки и в рабочей среде.

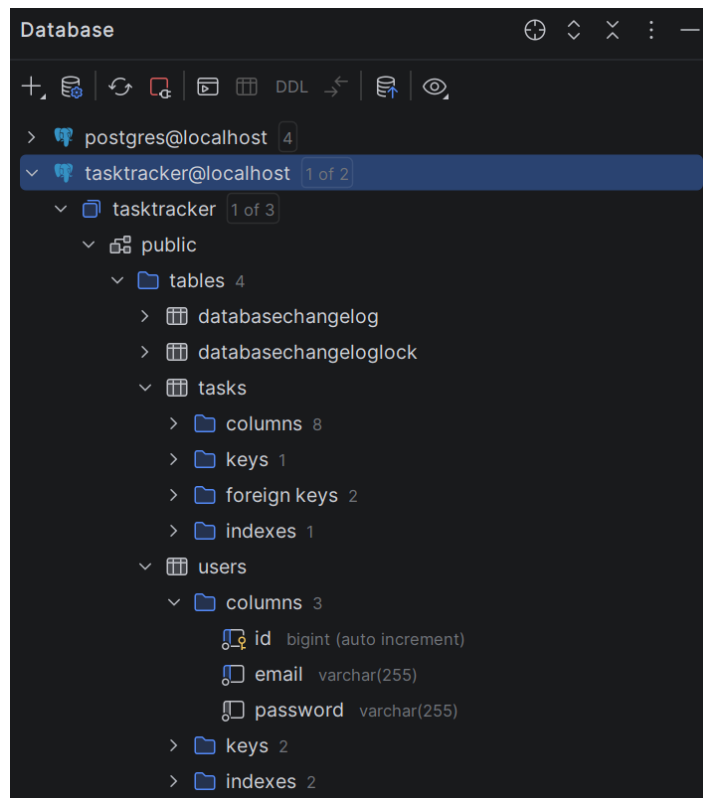


Рисунок 1 – Вкладка Database в IntelliJ IDEA, таблицы users, tasks

2.2 Система безопасности и авторизация (Spring Security + JWT)

Система безопасности спроектирована как Stateless (без хранения состояния на сервере).

Хеширование. Пароли пользователей не хранятся в открытом виде, используется алгоритм BCrypt (интерфейс PasswordEncoder), что защищает данные даже в случае утечки базы.

JWT Filter. Реализован пользовательский фильтр JwtAuthenticationFilter (наследник OncePerRequestFilter). Он извлекает токен из заголовка Authorization, проверяет его подпись и помещает данные пользователя в SecurityContextHolder.

Авторизация. Доступ к ресурсам /tasks/** закрыт для неавторизованных пользователей на уровне конфигурации SecurityFilterChain.

```
POST http://localhost:9090/login
```

```
HTTP/1.1 200
```

```
Authorization: Bearer
```

```
eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0QHRlc3QuY29tIiwiaWF0IjoxNzc5MjY3LCJleHAiOiE3Nz
```

```

kyODgwNjd9.rDatjwJzt_OW-0gLoQL1B0npq1Gc5BilS_5g3rZ5n8OnAwYrNARnA1sH-2D-vAoCdf-
rQaHb7Kwzv3BgdZushQ
X-Content-Type-Options: nosniff
X-XSS-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
X-Frame-Options: DENY
Content-Type: application/json
Transfer-Encoding: chunked
Date: Tue, 19 May 2026 14:41:07 GMT

{
  "token":
  "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJ0ZXN0QHRlc3QuY29tIiwiaWF0IjoxNzc5MjAxNjY3LCJleHAiOjE3N
  zkyODgwNjd9.rDatjwJzt_OW-0gLoQL1B0npq1Gc5BilS_5g3rZ5n8OnAwYrNARnA1sH-2D-vAoCdf-
  rQaHb7Kwzv3BgdZushQ"
}
Response file saved.
> 2026-05-19T204107.200.json

Response code: 200; Time: 793ms (793 ms); Content length: 196 bytes (196 B)

```

Листинг кода 2 – Логи ответа сервера на авторизацию

2.3 Реализация бизнес-логики управления задачами (Records, Switch expressions)

В сервисном слое (TaskService) реализованы механизмы проверки прав доступа: пользователь может редактировать и удалять только те задачи, владельцем которых он является. В ином случае система выбрасывает исключение `ResponseStatusException` со статусом 403 Forbidden.

Особое внимание уделено синтаксическим новшествам Java 21:

Records. DTO-классы (`AuthDto`, `TaskDto`) объявлены как рекорды. Это обеспечило автоматическую генерацию конструкторов, геттеров, методов `equals` и `hashCode`, сделав код лаконичным.

Switch Expressions. При обновлении статуса задачи использован синтаксис `switch`, который возвращает значение. Это позволило избежать громоздких конструкций `if-else` при определении даты завершения задачи (`doneAt`) в зависимости от выбранного статуса.

3 Дополнительные сервисы и интеграции

Современное веб-приложение должно не только отвечать на прямые запросы пользователя, но и обладать развитой системой уведомлений и автоматизации. В рамках данной главы рассматривается реализация почтовой подсистемы и механизмов планирования задач (Scheduling), которые расширяют базовый функционал трекера задач до уровня полноценного корпоративного инструмента.

3.1 Интеграция почтовой подсистемы на базе Mailpit

Для реализации требований по оповещению пользователей (приветственные письма и уведомления о назначении задач) была выбрана интеграция через протокол SMTP.

Специфика разработки и тестирования подобных систем требует наличия сервера, способного перехватывать исходящий трафик без риска отправки писем реальным адресатам. Для этих целей в стек технологий был включен Mailpit.

Реализация на Backend. Использован интерфейс `JavaMailSender`, входящий в состав `spring-boot-starter-mail`.

Логика работы. При регистрации нового пользователя метод `AuthService.register()` инициирует создание объекта `SimpleMailMessage`. Отправка происходит асинхронно по отношению к пользователю (благодаря обработке исключений), что гарантирует стабильность регистрации даже при временной недоступности почтового сервера.

Скриншот окна Mailpit в браузере (порт 8025), где видно список входящих писем «Welcome to Task Tracker» и «New Task Assigned» представлен на рисунке 2.

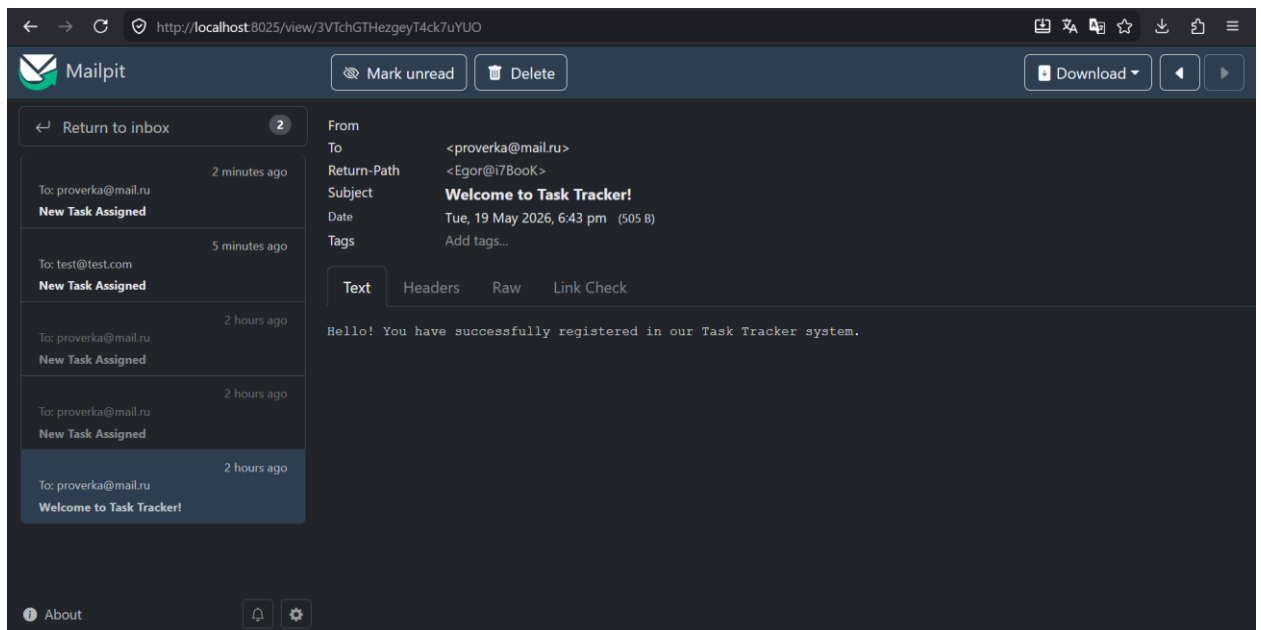


Рисунок 2 – Окно Mailpit в браузере (порт 8025), список входящих писем, «Welcome to Task Tracker» и «New Task Assigned»

3.2 Автоматизация фоновых процессов и планировщик (@Scheduled)

Согласно техническому заданию, система должна генерировать ежедневные отчеты. Для решения этой задачи был задействован механизм Spring Scheduling. В сервисе ReportService реализован метод sendDailyReports(), аннотированный @Scheduled(cron = "0 0 9 * * *").

Принцип работы. Каждое утро в 09:00 планировщик пробуждает метод, который выполняет агрегационный запрос к базе данных.

Алгоритм. Для каждого пользователя в системе подсчитывается количество задач в статусах WAITING и DONE. Сформированная статистика инкапсулируется в текстовое сообщение и отправляется пользователю. Использование Cron-выражений позволяет гибко настраивать периодичность отчетов без изменения программного кода.

3.3 Оркестрация сервисов через Docker Compose

Для обеспечения стабильного взаимодействия между бэкендом, базой данных PostgreSQL и почтовым сервером Mailpit использована

контейнеризация. В файле `docker-compose.yml` описана единая изолированная сеть, в которой компоненты обращаются друг к другу не по IP-адресам, а по именам сервисов (`postgres`, `mailpit`).

Это позволило изолировать среду разработки от конфигурации операционной системы и гарантировать наличие всех необходимых зависимостей при развертывании системы на новом сервере одной командой `docker compose up`.

Ниже представлен вывод по команде ***docker ps*** в терминале.

```
PS D:\PROJECTS\tasktracker> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS
NAMES
55773120f738   axllent/mailpit                    "/mailpit"              8 seconds ago Up 6
seconds (healthy) 0.0.0.0:1025->1025/tcp, [::]:1025->1025/tcp, 0.0.0.0:8025-
>8025/tcp, [::]:8025->8025/tcp task_mail
c9ec22efcf19   dpape/pgadmin4                    "/entrypoint.sh"        About an hour ago Up
About an hour    0.0.0.0:5050->80/tcp, [::]:5050->80/tcp
pgadmin
7dd639320520   postgres:16-alpine                "docker-entrypoint.s..." About an hour ago Up
About an hour    0.0.0.0:5434->5432/tcp, [::]:5434->5432/tcp
task_db
PS D:\PROJECTS\tasktracker>
```

Листинг кода 3 – Консоль с выводом по команде `docker ps`, где видны все три работающих контейнера `task_db`, `task_mail` и `pgadmin`

4 Клиентское приложение (Frontend)

В рамках работы было разработано клиентское приложение на базе архитектуры SPA (Single Page Application). Основной акцент при разработке был сделан на удобство пользователя (UX), высокую скорость отклика интерфейса и обеспечение безопасности при хранении данных сессии на стороне клиента.

4.1 Архитектура и стек технологий фронтенда

Для минимизации сложности сборки и обеспечения максимальной производительности интерфейс реализован на нативном JavaScript (ES6+) без использования тяжеловесных фреймворков.

Bootstrap 5 позволил создать адаптивную сетку, которая корректно отображается как на мониторах, так и на мобильных устройствах.

Bootstrap Icons использованы для интуитивно понятной визуализации действий (редактирование, удаление, смена статуса).

CSS3 Custom Styles применены для реализации эффектов при наведении (hover) и цветовой дифференциации статусов задач.

4.2 Механизм аутентификации и работа с JWT

Безопасность на стороне клиента организована следующим образом.

Хранение. После успешного выполнения запроса /login или /register, полученный от сервера JWT-токен сохраняется в LocalStorage браузера.

Передача. Для всех запросов к защищенным ресурсам (/tasks, /users, /me) используется встроенный перехватчик (через функцию getHeaders), который автоматически добавляет токен в заголовок Authorization: Bearer <token>.

Идентификация пользователя. При загрузке страницы приложение выполняет запрос к эндпоинту /me. Полученные данные (Email пользователя)

динамически вставляются в навигационную панель (Navbar), что позволяет пользователю всегда видеть, под какой учетной записью он авторизован.

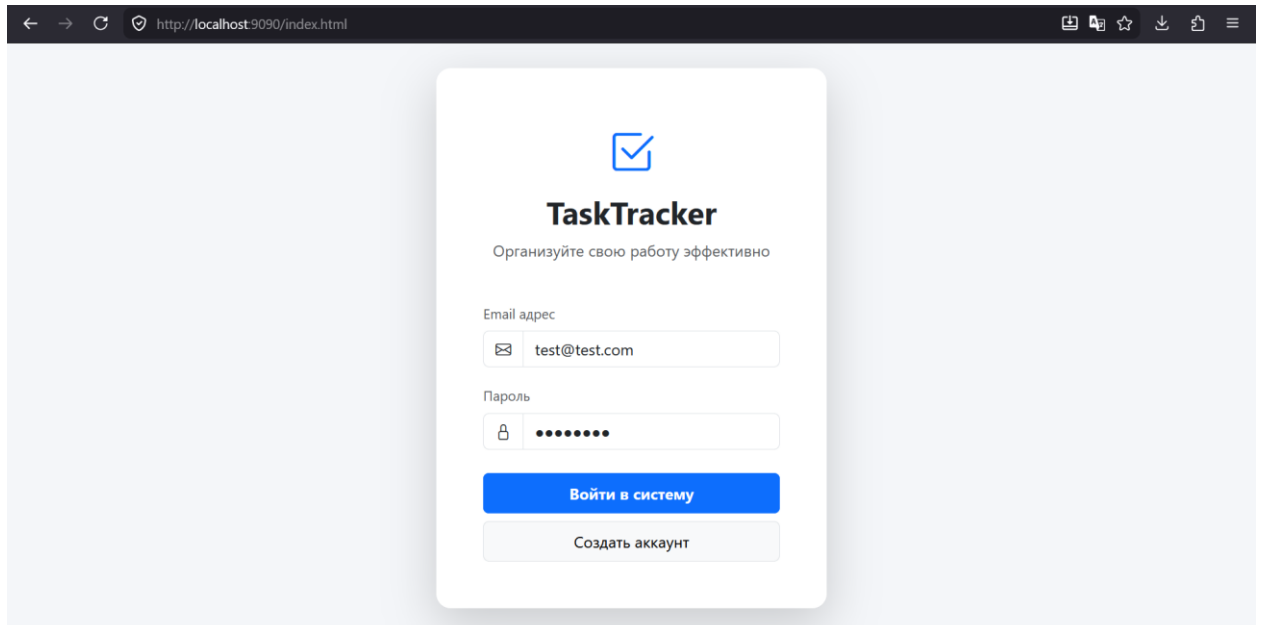


Рисунок 3 – Скриншот формы входа (Auth Card)

4.3 Функциональные возможности интерфейса

Интерфейс предоставляет пользователю полный цикл управления задачами (Full CRUD).

Добавление: Реализовано через верхнюю форму с поддержкой горячих клавиш (Enter).

Отображение. Задачи выводятся в виде интерактивных карточек. Выполненные задачи визуально отделяются (зачеркнутый текст, изменение цвета границы) и автоматически перемещаются в конец списка.

Редактирование и удаление. Вынесены в отдельные элементы управления с подтверждением действий (диалоговые окна), что минимизирует риск случайного удаления данных.

Назначение исполнителей. Реализовано через динамический выпадающий список (Select), который подгружает актуальный список всех зарегистрированных пользователей системы с бэкенда.

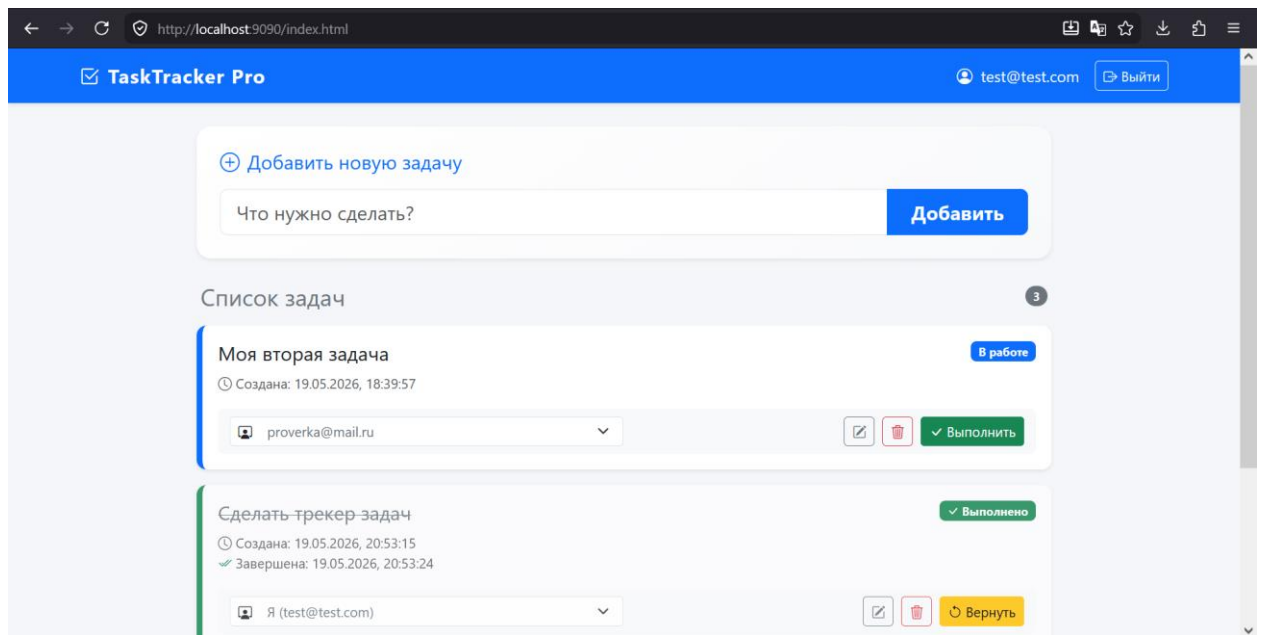


Рисунок 4 – Основной рабочий экран приложения. Navbar с почтой, список задач с иконками и выпадающий список выбора исполнителя

5 Тестирование и развертывание

Этап тестирования является критически важным для обеспечения надежности программного продукта. Было реализовано многоуровневое тестирование системы, включающее проверку бизнес-логики в изоляции и тестирование корректности работы REST-контроллеров.

5.1 Модульное и интеграционное тестирование (JUnit 5, Mockito)

Для верификации кода был использован современный стек инструментов тестирования.

JUnit 5 (Jupiter). Основной фреймворк для написания и запуска тестов.

Mockito. Библиотека для создания «моков» (имитаций) зависимостей. Это позволило протестировать TaskService без реального подключения к базе данных, имитируя поведение репозитория.

MockMvc. Инструмент для интеграционного тестирования веб-слоя. С его помощью была проверена работа эндпоинта /register, включая корректность обработки JSON и HTTP-статусов ответа.

5.2 Тестирование

В ходе работы было достигнуто глубокое покрытие кода тестами. Процесс верификации был разделен на две стратегии.

5.2.1 Unit-тестирование бизнес-логики (TaskServiceTest, AuthServiceTest)

Цель - проверка изолированных методов сервисов.

Методология. С помощью библиотеки Mockito были созданы дублиеры (моки) репозитория и почтового сервера. Это позволило проверить, что система корректно обрабатывает граничные случаи: например, выбрасывает исключение `ResponseStatusException` при попытке несанкционированного

доступа к чужой задаче или блокирует регистрацию на уже существующий email.

Результат - подтверждена корректность алгоритмов начисления даты завершения задач и механизмов шифрования паролей.

5.2.2 Интеграционное тестирование API (AuthControllerTest)

Цель - верификация «точек входа» приложения и работы фильтров безопасности.

Методология. Использование инструмента MockMvc позволило имитировать реальные HTTP-запросы (POST, GET, PATCH).

Ключевые проверки:

1. проверка эндпоинта /register: подтверждено, что сервер принимает JSON, корректно вызывает сервис и возвращает статус 200 OK;
2. проверка безопасности: подтверждено, что защищенный ресурс /tasks недоступен (статус 403 Forbidden) без предоставления валидного JWT-токена в заголовках.

Использование такого подхода гарантирует, что при внесении изменений в одну часть системы, например, изменение структуры базы данных, тесты укажут на возникшие ошибки в логике.

Результаты прохождения тестов представлены на рисунке ниже. Также сформированный отчет по тестам добавлен к корень проекта под именем «*Test Results - Tests_in_'tasktracker'.html*».

Tests in 'tasktracker': 8 total, 8 passed		6.23 s
		Collapse Expand
TasktrackerApplicationTests		2.35 s
contextLoads	passed	2.35 s
Интеграционное тестирование API		1.50 s
Точка входа /register: Успешный ответ	passed	1.49 s
Точка входа /tasks: Должна возвращать 403 без токена	passed	12 ms
Тестирование аутентификации (AuthService)		1.44 s
Регистрация: ошибка при занятом Email	passed	1.44 s
Тестирование логики управления задачами (TaskService)		939 ms
Создание задачи: проверка корректности заполнения полей	passed	855 ms
Назначение исполнителя: проверка отправки уведомления	passed	39 ms
Безопасность: запрет на удаление чужой задачи	passed	21 ms
Обновление статуса: переход в DONE должен ставить дату завершения	passed	24 ms

Generated by IntelliJ IDEA on 19.05.2026, 21:45

Рисунок 5 – Отчет по тестам

5.3 Процесс сборки и контейнеризация (Docker)

Развертывание системы автоматизировано с помощью Docker Desktop. Проект собирается в исполняемый JAR-файл с использованием Gradle.

С помощью Docker Compose все компоненты (приложение, БД PostgreSQL, почтовый клиент Mailpit) объединяются в единую инфраструктуру. Это гарантирует, что система будет работать одинаково на любом сервере, где установлен Docker, независимо от версии, установленной Java или настроек локальной СУБД. Скриншот из приложения Docker Desktop представлен на рисунке 6.

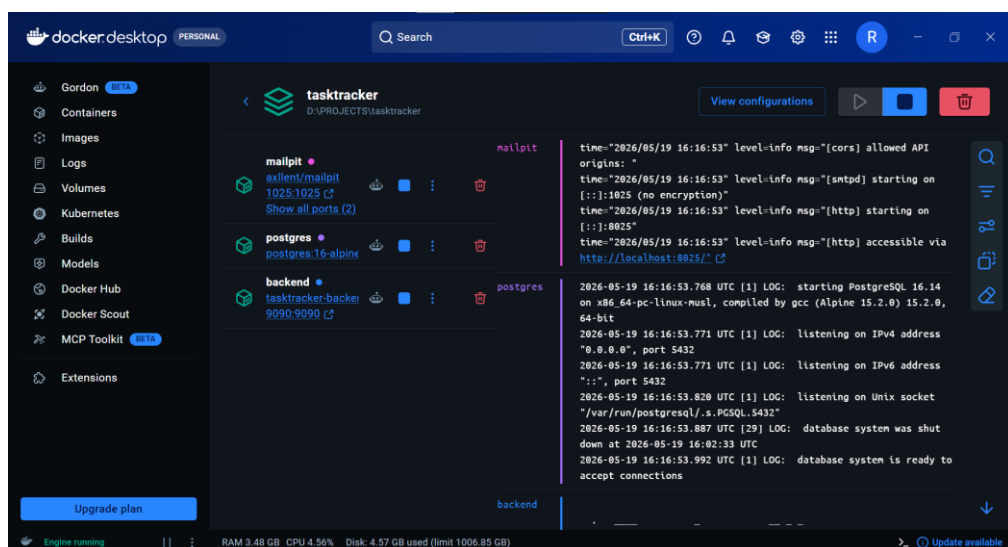


Рисунок 6 – скриншот из приложения Docker Desktop, где видны все три работающих контейнера task_db, task_mail и pgadmin

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной работы было разработано полнофункциональное веб-приложение «Трекер задач». В процессе реализации были решены следующие задачи:

1. спроектирована и реализована архитектура бэкенда на базе Spring Boot 4 и Java 21;
2. организована надежная система хранения данных с использованием PostgreSQL и автоматических миграций Liquibase;
3. реализована современная система безопасности на базе JWT-токенов;
4. разработано клиентское приложение (SPA), обеспечивающее удобное взаимодействие пользователя с системой;
5. интегрированы дополнительные сервисы: почтовая рассылка через Mailpit и планировщик отчетов.

Результатом работы является готовый к развертыванию сервис, соответствующий современным стандартам веб-разработки. В ходе практики были закреплены навыки работы с контейнеризацией, современным синтаксисом Java и проектированием RESTful API.